

11.3 Reallocating Memory Blocks

Sometimes you're not sure at first how much memory you'll need. For example, if you need to store a series of items you read from the user, and if the only way to know how many there are is to read them until the user types some ``end" signal, you'll have no way of knowing, as you begin reading and storing the first few, how many you'll have seen by the time you do see that ``end" marker. You might want to allocate room for, say, 100 items, and if the user enters a 101st item before entering the ``end" marker, you might wish for a way to say ``uh, malloc, remember those 100 items I asked for? Could I change my mind and have 200 instead?"

In fact, you can do exactly this, with the `realloc` function. You hand `realloc` an old pointer (such as you received from an initial call to `malloc`) and a new size, and `realloc` does what it can to give you a chunk of memory big enough to hold the new size. For example, if we wanted the `ip` variable from an earlier example to point at 200 `ints` instead of 100, we could try calling

```
ip = realloc(ip, 200 * sizeof(int));
```

Since you always want each block of dynamically-allocated memory to be contiguous (so that you can treat it as if it were an array), you and `realloc` have to worry about the case where `realloc` can't make the old block of memory bigger ``in place," but rather has to relocate it elsewhere in order to find enough contiguous space for the new requested size. `realloc` does this by returning a new pointer. If `realloc` was able to make the old block of memory bigger, it returns the same pointer. If `realloc` has to go elsewhere to get enough contiguous memory, it returns a pointer to the new memory, after copying your old data there. (In this case, after it makes the copy, it frees the old block.) Finally, if `realloc` can't find enough memory to satisfy the new request at all, it returns a null pointer. Therefore, you usually don't want to overwrite your old pointer with `realloc`'s return value until you've tested it to make sure it's not a null pointer. You might use code like this:

```
int *newp;
newp = realloc(ip, 200 * sizeof(int));
if(newp != NULL)
    ip = newp;
else
    {
        printf("out of memory\n");
        /* exit or return */
        /* but ip still points at 100 ints */
    }
```

If `realloc` returns something other than a null pointer, it succeeded, and we set `ip` to what it returned. (We've either set `ip` to what it used to be or to a new pointer, but in either case, it points to where our data is now.) If `realloc` returns a null pointer, however, we hang on to our old pointer in `ip` which still points at our original 100 values.

Putting this all together, here is a piece of code which reads lines of text from the user, treats each line as an integer by calling `atoi`, and stores each integer in a dynamically-allocated ``array":

```
#define MAXLINE 100

char line[MAXLINE];
int *ip;
int nalloc, nitems;

nalloc = 100;
ip = malloc(nalloc * sizeof(int));          /* initial allocation */
if(ip == NULL)
    {
```

```

        printf("out of memory\n");
        exit(1);
    }

nitems = 0;

while(getline(line, MAXLINE) != EOF)
{
    if(nitems >= nalloc)
    {
        /* increase allocation */
        int *newp;
        nalloc += 100;
        newp = realloc(ip, nalloc * sizeof(int));
        if(newp == NULL)
        {
            printf("out of memory\n");
            exit(1);
        }
        ip = newp;
    }

    ip[nitems++] = atoi(line);
}

```

We use two different variables to keep track of the ``array" pointed to by `ip`. `nalloc` is how many elements we've allocated, and `nitems` is how many of them are in use. Whenever we're about to store another item in the ``array," if `nitems >= nalloc`, the old ``array" is full, and it's time to call `realloc` to make it bigger.

Finally, we might ask what the return type of `malloc` and `realloc` is, if they are able to return pointers to `char` or pointers to `int` or (though we haven't seen it yet) pointers to any other type. The answer is that both of these functions are declared (in `<stdlib.h>`) as returning a type we haven't seen, `void *` (that is, pointer to `void`). We haven't really seen type `void`, either, but what's going on here is that `void *` is specially defined as a ``generic" pointer type, which may be used (strictly speaking, assigned to or from) any pointer type.

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995-1997 // [mail feedback](#)